

Quick Sort Algorithm: A Structured Note

1 Introduction and Core Paradigm

- Quick Sort is an algorithm built upon the **divide and conquer** strategy [1, 2].
- The fundamental idea is to divide an array into two parts, solve (sort) each part recursively, and put the solutions back together to sort the entire array [1, 3].
- Unlike Merge Sort, where dividing the array is trivial and the bulk of the computational work is spent “putting together” (merging), Quick Sort takes the exact opposite approach [4]. In Quick Sort, the algorithm spends most of its time on the “divide” step, making the final step of putting the array back together completely trivial [4, 5].

2 The Algorithm Steps

The Quick Sort algorithm can be summarized in three steps:

- **Pick a pivot:** Choose an element from the array, which will be called the pivot (p) [4, 6].
- **Divide into two parts:** Rearrange the array elements so that all elements smaller than p are placed in one part, and all elements greater than (or equal to) p are placed in the other part [4, 6].
- **Recurse and put together:** Recursively apply this process to the two parts [4]. Because the left portion is completely sorted and every element in it is smaller than every element in the sorted right portion, the entire array is inherently sorted when put together [2, 5]. No additional work is required in this final step [5].

3 The Concept of “Rank”

To understand the performance of Quick Sort, it is essential to understand the “rank” of the chosen pivot [7].

- **Definition:** The rank of an element is its exact position in the final sorted order of the array [7].
- Assuming all elements in the array are distinct, each element has a distinct rank [7].
- If there are i elements smaller than a chosen element x , the rank of x is $i + 1$ [7]. The minimum element in the array has a rank of 1, and the maximum element has a rank of n (the size of the array) [7].

4 Time Complexity Analysis

The time Quick Sort takes to run depends entirely on the rank of the element picked as the pivot [4, 7].

- To divide the array, the algorithm compares each element with the pivot to determine if it goes to the left or the right [8]. This requires $n - 1$ comparisons, meaning the time taken to partition the array is roughly n [8].
- If you pick a pivot with rank i , the array splits into one side containing $i - 1$ elements and another side containing $n - i$ elements [7].
- This gives us the generic recurrence relation for the algorithm's time:

$$T(n) \leq T(i - 1) + T(n - i) + n$$

[8].

From this relation, the algorithm's running time varies based on the pivot chosen:

4.1 The Worst-Case Scenario ($O(n^2)$)

- This occurs if the algorithm repeatedly picks the minimum element (or the maximum element) as the pivot at each step [8–10].
- If the minimum element is picked, there are zero elements on the left and $n - 1$ elements on the right [9].
- The recurrence relation becomes $T(n) = T(n - 1) + n$ [9].
- Expanding this mathematically yields $1 + 2 + \dots + n$, which equals $\frac{n(n-1)}{2}$ [9]. This results in an overall worst-case running time of $O(n^2)$ [9, 10].

4.2 The Best-Case Scenario ($O(n \log n)$)

- This occurs if you manage to pick roughly the middle element (an element of rank $\approx n/2$) as the pivot [10].
- This divides the array into two subproblems of roughly equal size [10].
- The recurrence relation becomes $T(n) = 2T(n/2) + n$ [10].
- This perfectly balanced recurrence evaluates to a highly efficient running time of $O(n \log n)$ [10].

5 Algorithm Optimization: Finding the Median

Because the standard implementation of Quick Sort doesn't know the rank of a pivot beforehand, the running time risks falling into the $O(n^2)$ worst-case [11].

- In the divide and conquer strategy, it is always desirable to split the input into two exactly equal halves, as uneven splits proportionally increase the time taken [12].
- To guarantee an $O(n \log n)$ running time, we must efficiently pick an element of rank roughly $n/2$ (the median) to use as the pivot [11–13].
- The median can be defined as the element with rank $\lfloor n/2 \rfloor$ or $\lceil n/2 \rceil$ [13].

- While you could easily find the median by sorting the array first (which takes $O(n \log n)$ time), this is not fast enough [13]. We need an algorithm that can find the median strictly in **linear time** ($O(n)$) [12, 13].
- If the median can be found in linear time, the recurrence remains $T(n) = 2T(n/2) + C \cdot n$ (where C is a constant), which still securely yields an overall Quick Sort time of $O(n \log n)$ [11].